

Séquenceur février

Conformément à la conclusion du rapport de janvier, l'étape suivante prévue au cahier des charges était la persistance des données. Le mois de février a donc été intégralement consacré à l'implémentation de la sauvegarde et du chargement des projets au format JSON, afin de permettre à l'utilisateur de conserver ses créations et compositions. Le moteur audio asynchrone et l'interface étant désormais stables, cette étape était cruciale pour transformer le prototype en un véritable logiciel utilitaire.

Système de charge / sauvegarde

Pour répondre à cette exigence technique, j'ai fait le choix d'utiliser le format JSON, facilement manipulable en Python avec la bibliothèque standard json. Au niveau de la logique métier, j'ai enrichi la classe SequenceurCore de mon architecture avec deux nouvelles méthodes de sérialisation et désérialisation (exporter_donnees et importer_donnees). Lors d'une sauvegarde, le programme extrait l'état complet du séquenceur sous forme de dictionnaire : le tempo actuel (BPM), le volume global, ainsi que l'état précis de la grille de 16 pas et du mode "Mute" pour chaque piste.

Évolution de l'interface graphique et synchronization

Sur le plan de l'interface graphique, j'ai intégré une barre de contrôles dédiée à la gestion des fichiers en haut de la fenêtre principale. L'utilisation de la classe QFileDialog issue de la bibliothèque PySide6 a permis de déclencher l'ouverture de boîtes de dialogue natives offrant une expérience utilisateur familière et fluide.

Le défi technique principal de ce mois a résidé dans la synchronisation visuelle lors du chargement d'un projet existant. Il a fallu développer une logique permettant de mettre à jour dynamiquement tous les composants visuels de l'interface de manière programmatique :

Ajustement automatique de la position des sliders de Tempo et de Volume en fonction des données chargées.

Mise à jour visuelle des cases à cocher de la matrice de 16 pas pour chaque instrument, afin de refléter fidèlement le rythme sauvegardé.

Application immédiate des paramètres temporels au QTimer sans perturber le moteur audio en cours de route.

Conclusion

Les objectifs fixés pour le mois de février sont pleinement validés. Le logiciel dispose désormais d'un système de sauvegarde et de chargement robuste, permettant aux utilisateurs de ne plus perdre leur travail à la fermeture de l'application. Des tests de robustesse ont été effectués (chargement de rythmes très rapides à 180 BPM en cours de lecture) et le moteur développé en janvier supporte parfaitement ces transitions dynamiques.

Le mois de mars sera dédié à l'implémentation de fonctionnalités avancées qui enrichiront l'expérience musicale. Il est notamment prévu d'ajouter un explorateur de fichiers permettant à l'utilisateur de charger et d'importer ses propres samples audio (.wav) pour s'affranchir des sons par défaut, et de commencer à étudier l'exportation du mixage final en fichier audio.